

5 Adaptive Filtering

5.1 Introduction to linear adaptive filtering

- Multivariate regression of **non-stationary**, stochastic processes
- Given: ⁽¹⁾

- Sought: ⁽²⁾

n is the original domain index (discrete time or space index)

- **Filtering** is an **on-line** procedure to generate an **output** value $y(n)$ from current and past **input** values $x(n), x(n-1), x(n-2), \dots$ to estimate a **target** value $t(n)$

Example: adaptive control, $x(n)$ are observations of pressure sensors, $t(n)$ is volume in a water tank.

- **Smoothing** is an **off-line** procedure which processes stored data $x(n)$ and aims to generate $y(n)$ which has lower noise and / or lower amount of fluctuations

Example: adaptive equalizer, $x(n)$ is a music signal, $t(n)$ is a target music signal with emphasis in distinct frequency bands and suppression in other frequency bands.

- **Prediction** is an **on-line** procedure to estimate $y(n)$ which should be close to future values $t(n) = x(n+h)$, where h is the prediction horizon.

Example: stock price forecast, $x(n)$ are current and past observations, $t(n) = x(n+h)$ are known future observations; they can only be obtained from existing datasets.

- Four types of applications

1. **System identification** ⁽³⁾

The filter is adapted to a
SUT (system under test)
which operates in parallel.

2. **Inverse modelling** ⁽⁴⁾

The filter is adapted to a
SUT (system under test)
which operates in series.

3. **Prediction** ⁽⁵⁾

The filter is adapted such that
the output signal is close to
future samples of the input signal.

4. **Interference cancellation** ⁽⁶⁾

The filter processes an input signal which is
interfered by other disturbing components.

The filter is adapted such that the output
is close to a reference signal.

Typical examples are

- Adaptive noise cancellation
- Echo cancellation.

Signal diagram ⁽⁷⁾

of an adaptive filter

- Principle:
error correction

Update of filter weight vector, i.e. weight series of a discrete LTI system

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \Delta \mathbf{w}(n) \quad (5.1)$$

- Whereby the filter order is p and the current **p -tap input vector** $\mathbf{x}(n)$ and the current filter weight vector $\mathbf{w}(n)$ are defined as follows:

$$\mathbf{x}(n) = (x(n), x(n-1), \dots, x(n-p+1))^T, \quad \mathbf{w} = (w_0(n), w_1(n), \dots, w_{p-1}(n))^T \quad (5.2)$$

- Their scalar product $\mathbf{w}(n) \cdot \mathbf{x}(n)$ represents the **discrete convolution** of $\mathbf{w}$ and $\mathbf{x}$:

$$\mathbf{w}(n) * \mathbf{x}(n) = \sum_{m=0}^{p-1} w_m(n) x(n-m) = \mathbf{w}(n) \cdot \mathbf{x}(n) \quad (5.3)$$

- Thus, (5.3) gives the actual filter output $y(n)$. The current error value $e(n)$ is defined as the difference between filter output and the target value $t(n)$:

$$e(n) = t(n) - y(n) = t(n) - \mathbf{w}(n) \cdot \mathbf{x}(n) \quad (5.4)$$

- The objective function $J(n)$ to be minimised is the ensemble averaged, squared error:

$$J(n) = \frac{1}{2} \mathbb{E}[e^2(n)] \quad (5.5)$$

- To find the minimum of (5.5), the first derivate to all weights is needed, i.e. the **gradient**:

$$\frac{\partial J(n)}{\partial \mathbf{w}(n)} = \frac{1}{2} \frac{\partial \mathbb{E}[e^2(n)]}{\partial \mathbf{w}(n)} = \frac{1}{2} \mathbb{E} \left[\frac{\partial e^2(n)}{\partial \mathbf{w}(n)} \right] = \mathbb{E} \left[e(n) \frac{\partial e(n)}{\partial \mathbf{w}(n)} \right] = - \mathbb{E}[e(n) \mathbf{x}(n)] \quad (5.6)$$

- The weight vector is updated in order to go iteratively to the minimum. The weight vector is moved step by step in the opposite direction of the gradient, i.e. **gradient descent** with step size η :

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta \frac{\partial J(n)}{\partial \mathbf{w}(n)} = \mathbf{w}(n) + \eta \mathbb{E}[e(n) \mathbf{x}(n)] \quad (5.7)$$

5.1.1 LMS algorithm (least mean square)

- Under the assumption of normal distributed variables, the expectation operator in (5.7) is approximated by the arithmetic mean of the current and the last N samples:

$$\hat{\mathbb{E}}[e(n) \mathbf{x}(n)] = \frac{1}{N} \sum_{i=0}^{N-1} e(n-i) \mathbf{x}(n-i) \quad (5.8)$$

- The simplest case is to set $N = 1$, i.e. only the current sample defines the expected error:

$$\hat{\mathbb{E}}[e(n) \mathbf{x}(n)] = e(n) \mathbf{x}(n) \quad (5.9)$$

- Convergence properties

- Optimal weights $\mathbf{w}(n)$ are not guaranteed because $\mathbb{E}(e(n)\mathbf{x}(n))$ is unknown
- Approximation $\hat{\mathbb{E}}(\cdot)$ leads to convergence in the mean: estimated $\mathbf{w}(n)$ are varying about the optimum
- But variance is not guaranteed to be small
- The **step size** η is a critical parameter and must be chosen properly to be efficient
 - η too small $\rightarrow$ low speed of convergence, good choice for noisy signals
 - η too large $\rightarrow$ increased risk of divergence of $J(n)$, in case of very low noise
- An **upper bound** has been proposed: $0 < \eta < \frac{2}{\lambda_{max}}$, where λ_{max} is the largest eigenvalue of the auto-correlation matrix $\mathbf{R}_{xx} = \mathbb{E}(\mathbf{x}(n)\mathbf{x}^T(n))$
- **Convergence speed** depends on η
 - Maximum speed if the following is fulfilled: $\eta = \frac{2}{\lambda_{max} + \lambda_{min}}$
- **Eigenvalue spread** determines speed
 - White noise signal $\rightarrow \mathbf{R}_{xx} = \sigma^2 \mathbf{I} \rightarrow$ all eigenvalues are equal $\rightarrow$ zero spread $\rightarrow$ maximum speed
 - Signals with nonuniform spectrum $\rightarrow$ high spread $\rightarrow$ lower speed compared to white noise at input
- Alternative bound with trace of $\mathbf{R}$: it guarantees that weight vectors $\mathbf{w}(n)$ do not diverge: $\eta < \frac{2}{tr(\mathbf{R}_{xx})}$
- **Normalised LMS**: algorithm is insensitive to scaling of the p -tap input signal $\mathbf{x}(n)$
 - Normalisation by signal power: $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta \frac{e(n) \mathbf{x}(n)}{\mathbf{x}(n) \cdot \mathbf{x}(n)}$

- Summary of the LMS algorithm

- Define constants: ⁽¹⁾ p, η
- Initialisations: ⁽²⁾ $\mathbf{w}(0) = \mathbf{0} = (0, 0, \dots, 0)^T$
- Calculations: ⁽³⁾ *for* $n = 0, 1, 2, \dots$

$$\mathbf{x}(n) = (x(n), x(n-1), \dots, x(n-p+1))^T$$

$$e(n) = t(n) - \mathbf{w}(n) \cdot \mathbf{x}(n)$$

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta e(n) \mathbf{x}(n)$$

end

- LMS algorithm does not depend on the model: $y(n) = \mathbf{w}(n) \cdot \mathbf{x}(n)$
- LMS is a robust algorithm
small fluctuations in the input signal $x(n)$ do not induce large fluctuations in the output signal $y(n)$

5.1.2 RLMS algorithm (recursive least mean squares)

- Minimisation of weighted LMS objective function for all time steps $i = 0, \dots, n$ with λ as forgetting factor

$$J(n) = \frac{1}{2} \sum_{i=0}^n \lambda^{n-i} e^2(i) \quad (5.10)$$

- For the gradient of the objective function it follows:

$$\frac{\partial J(n)}{\partial \mathbf{w}(n)} = \sum_{i=0}^n \lambda^{n-i} e(i) \frac{\partial e(i)}{\partial \mathbf{w}(n)} = - \sum_{i=0}^n \lambda^{n-i} e(i) \mathbf{x}(i-k) \quad (5.11)$$

- In the minimum the gradient (5.11) must vanish:

$$\sum_{i=0}^n \lambda^{n-i} (t(i) - \hat{\mathbf{w}}(i) \cdot \mathbf{x}(i)) \mathbf{x}(i-k) = 0 \quad (5.12)$$

$$\sum_{i=0}^n \lambda^{n-i} t(i) \mathbf{x}(i-k) = \sum_{i=0}^n \lambda^{n-i} \left(\sum_{k=0}^{p-1} \hat{\mathbf{w}}_k(i) x_k(i) \right) \mathbf{x}(i-k)$$

$$\hat{\mathbf{r}}_{TX}(n, \lambda) = \hat{\mathbf{R}}_{XX}(n, \lambda) \hat{\mathbf{w}}(n)$$

- Left-hand side equals the weighted cross-covariance between $t(n)$ and $x(n)$: $\hat{\mathbf{r}}_{TX}(n, \lambda)$
- Right-hand side equals the product of sample covariance matrix and weight vector: $\hat{\mathbf{R}}_{XX}(n, \lambda) \hat{\mathbf{w}}(n)$
- The solution is found by inverting (5.12)

$$\hat{\mathbf{w}}(n) = \hat{\mathbf{R}}_{XX}^{-1}(n, \lambda) \hat{\mathbf{r}}_{TX}(n, \lambda) \quad (5.13)$$

- Forgetting factor λ

- small $\lambda \rightarrow$ high forgetting
small window of data contributing to $\mathbf{R}_{XX}(n) \rightarrow$ filter is more sensitive to recent samples
- large $\lambda \rightarrow$ low forgetting
maximum: $\lambda = 1 \rightarrow$ growing window RLS algorithm
- oftentimes λ is fixed to be in a small interval: $\lambda \in (0.98, 1.00)$

- Recursions

$$\hat{\mathbf{r}}_{TX}(n, \lambda) = \sum_{i=0}^n \lambda^{n-i} t(i) \mathbf{x}(i) = \sum_{i=0}^{n-1} \lambda^{n-i} t(i) \mathbf{x}(i) + \lambda^0 t(n) \mathbf{x}(n) \quad (5.14)$$

$$\hat{\mathbf{r}}_{TX}(n, \lambda) = \hat{\mathbf{r}}_{TX}(n-1, \lambda) + t(n) \mathbf{x}(n)$$

$$\hat{\mathbf{R}}_{XX}(n, \lambda) = \sum_{i=0}^n \lambda^{n-i} \mathbf{x}(i) \mathbf{x}^T(i) = \sum_{i=0}^{n-1} \lambda^{n-i} \mathbf{x}(i) \mathbf{x}^T(i) + \lambda^0 \mathbf{x}(n) \mathbf{x}^T(n) \quad (5.15)$$

$$\hat{\mathbf{R}}_{XX}(n, \lambda) = \hat{\mathbf{R}}_{XX}(n-1, \lambda) + \mathbf{x}(n) \mathbf{x}^T(n)$$

- A recursion for the inverse matrix can be found after some algebraic rearrangements:

$$\hat{\mathbf{R}}_{XX}^{-1}(n, \lambda) = \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) - \mathbf{g}(n) \mathbf{x}^T(n) \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) \quad (5.16)$$

- The **Kalman gain vector** $\mathbf{g}(n)$ is calculated as follows:

$$\mathbf{g}(n) = \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) \mathbf{x}(n) \left(1 + \lambda^{-1} \mathbf{x}^T(n) \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) \mathbf{x}(n) \right)^{-1} \quad (5.17)$$

- After further algebraic rearrangements, the following recursion can be made for the filter weight vector:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \tilde{\mathbf{e}}(n) \mathbf{g}(n) \quad (5.18)$$

- With the so-called **a-priori estimation error**: $\tilde{\mathbf{e}}(n) = t(n) - \mathbf{w}(n-1) \cdot \mathbf{x}(n) \quad (5.19)$

- Instead of the **a-posteriori estimation error**: $\mathbf{e}(n) = t(n) - \mathbf{w}(n) \cdot \mathbf{x}(n) \quad (5.20)$

- This determines all calculations. The algorithm must be processed in the following steps.

- Summary of the RLS algorithm

- Define constants: model order p , forgetting factor λ , initial variance estimation δ

- Initialisations: ⁽³⁾ $\mathbf{w}(0) = \mathbf{0} = (0, \dots, 0)^T, \mathbf{x}(k) = \mathbf{0}, t(k) = 0 \quad \forall k = -p, \dots, 1; \hat{\mathbf{R}}_{XX}^{-1}(0) = \delta \mathbf{I}$

- Calculations: for $n = 1, 2, \dots$

$$(5.2) \quad \mathbf{x}(n) = (x(n), x(n-1), \dots, x(n-p+1))^T$$

$$(5.19) \quad \tilde{\mathbf{e}}(n) = t(n) - \mathbf{w}(n-1) \cdot \mathbf{x}(n)$$

$$(5.17) \quad \mathbf{g}(n) = \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) \mathbf{x}(n) \left(1 + \lambda^{-1} \mathbf{x}^T(n) \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) \mathbf{x}(n) \right)^{-1}$$

$$(5.16) \quad \hat{\mathbf{R}}_{XX}^{-1}(n, \lambda) = \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda) - \mathbf{g}(n) \mathbf{x}^T(n) \lambda^{-1} \hat{\mathbf{R}}_{XX}^{-1}(n-1, \lambda)$$

$$(5.18) \quad \mathbf{w}(n) = \mathbf{w}(n-1) + \tilde{\mathbf{e}}(n) \mathbf{g}(n)$$

end for

- General remarks

- Weight sequence $\mathbf{w}(n-1)$ provides a way of processing all data from $i = 0$ up to $i = n$

- Complexity of RLS: $\mathcal{O}(n^2)$ complexity of LMS: $\mathcal{O}(n)$

- Advantages of RLS: Faster convergence; one order of magnitude faster

- Drawback of RLS: less robust; more complex algorithm (RLS introduces an inverse auto-correlation matrix with stepwise update; LMS has only one simple scalar error variable)

- More details can be found in: Haykin S (2002) *Adaptive filter theory*. 4th edition. Prentice Hall

5.2 Nonlinear adaptive filtering

- **Linear mappings** of input to output signals are oftentimes **not sufficient** in practise
- **Sought:** continuous nonlinear function based on given discrete samples
- SRM: expected loss must be minimised by a function as simple as possible, but explaining the data
If the function was chosen too simple, then local minima result during training
- **Solutions alternative to nonlinear kernel adaptive filtering**
 - a) Simple solution: cascade of a linear filter with static nonlinearity
 - b) Volterra series
complexity explodes exponentially with increasing model complexity
 - c) Artificial neural networks (ANN): stacked MLP nets, RBF nets, RNN
successful; but non-convex optimisation → efficiency is low, no online applications
 - d) Recursive Bayes estimation: estimate unknown pdf; recursively with successive measurements and with mathematical model of the process; closed link to Kalman filter
 - e) Reinforcement learning: immediate evaluation with praise and admonition (1/0) after update, but no information on direction of the next update → online application is crucial

5.2.1 Kernel adaptive filtering

- Adaptive filtering within the **reproducing kernel Hilbert space** (RKHS), see chapter 6
 - RKHS is crucial for properties like: linearity, convexity, universal approximation
- **Properties**
 - Universal approximation (every problem class is solvable)
 - No local minima during optimization of free parameters, always convex
 - Moderate complexity (CPU time, working memory)
 - Continuous in- / output signal mappings → multivariate regression: $y = h(\mathbf{x})$
 - Recursive learning: $h_n = h_{n-1} + \mathbf{w}(n) \cdot \mathbf{g}(n) e(n)$ with gain vector and innovation
 - Kernel function methods are memory intensive
- **Approach**
 - Transformation of input vectors to the high-dimensional RKHS by reproducing kernel functions
 - Utilization of inner products $\mathbf{f}(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x}') = \kappa(\mathbf{x}, \mathbf{x}')$ which are efficiently computable
 - The mapping vectors $\mathbf{f}(\mathbf{x})$ itself do not have to be computed within the high-dimensional RKHS
- **Historic timeline** of adaptive kernel filtering
 - Volterra and Wiener series are special cases of approximations in RKHS

- 1999: kernel Adaline [Frieb & Harrison]	- 2008: kernel affine projection [Liu & Principe]
- 2004: kernel leaky LMS [Kivinen et al.]	- 2008: affine kernel proj. [Slavakis & Theodoridis]
- 2004: kernel RLS [Engel et al.]	- 2009: affine kernel projection [Richard et al.]
- 2008: kernel LMS [Liu et al.]	- 2009: kernel extended RLS [Liu et al.]
- **Criteria of model selection**
 - Evaluation of the quality of model fitted to a given data set. **Minimization** is searched for:
 - 1) Akaike information criterion [1974] (k = number of free parameters, L = likelihood function)

$$g_{AIC} = 2k + 2N \ln(MSE) = 2k - 2 \ln(L_{max}) \quad (5.21)$$

(*) if residuals are i.i.d. and n.d.
 - 2) Bayes information criterion [Schwarz, 1978]

$$g_{BIC} = k \ln(N) + N \ln(MSE) = k \ln(N) - 2 \ln(L_{max}) \quad (5.22)$$
 - 3) Minimum description length (MDL) → formalisation of sparseness principle (Occam's razor)
→ estimation of the Kolmogorov complexity
 - **Sparsification** is oftentimes meaningful to reduce the size of data set used for building a model
 - Selection of meaningful data: e.g. by distance-based data selection (see below when introducing a set C), by vector quantisation to find prototype vectors, or by searching for support vectors (see ch. 6)
 - Demand for **convergence in the mean** is too weak for practical applications:

$$\mathbb{E}(\mathbf{w}(n)) = \mathbf{w}^* \text{ for } n \rightarrow \infty, N \rightarrow \infty \quad (5.23)$$

- Demand for **convergence in the quadratic mean** holds if $0 < \eta < 1/\xi_{\max}$ is fulfilled, where $\xi_{\max}$ is the largest eigenvalue of the auto-covariance matrix $\hat{\mathbf{R}}_{XX} = \sum_{n=1}^N \mathbf{x}(n)\mathbf{x}^T(n)$:

$$\mathbb{E}(\mathbf{w}^2(n)) = \text{const} \text{ for } n \rightarrow \infty, N \rightarrow \infty \quad (5.24)$$

- Conservative estimation of the largest eigenvalue: $\xi_{\max} \approx \text{tr}(\hat{\mathbf{R}}_{XX})$

- Misadjustment criterion

$$m = \frac{J(\infty) - J_{\min}}{J_{\min}}, \quad J(\infty) = \lim_{n \rightarrow \infty} \mathbb{E}(e^2(n)), N \rightarrow \infty \quad (5.25)$$

- $J_{\min}$ is the non-reducible error rate caused by failing model adaptation and observation noise
- For low step size η the misadjustment can be approximated as follows:

$$m \approx \frac{1}{2} \eta \sum_{i=1}^n \xi_i \approx \frac{1}{2} \eta \text{tr}(\mathbf{R}_{XX}(n)) \quad (5.26)$$

- $m \leq 0.1$ often yields a relatively good approximation
- Learning performance plot
 - Ensemble mean: group of filters of the same architecture, but different parameters
 - Test approximation by repeated partitioning into trainings and test set

5.2.2 Kernel function LMS (KLMS)

- Adaptation of input-output mappings $h: \mathcal{X} \rightarrow \mathbb{R}$
based on a training set: $\{(\mathbf{x}(1), t(1)), (\mathbf{x}(2), t(2)), \dots, (\mathbf{x}(N), t(N))\}, \mathbf{x} \in \mathcal{X} \subset \mathbb{R}^d, t \in \mathbb{R}$
- $\mathcal{X}$ is a subset (region) of the vector space $\mathbb{R}^d$, the elements $\mathbf{x}$ are feature vectors or p -tap input vectors
- Assumptions
 - Generalised linear FIR filter
 - Using kernel functions for nonlinear mapping to the high-dimensional transform space with $\mathbf{f}(\mathbf{x}(n))$
 - Generalised linear mapping $\mathbf{w} \cdot \mathbf{f}(\mathbf{x})$ is much more powerful as $\mathbf{w} \cdot \mathbf{x}$, because there exist richer forms of representation and much higher degree of freedom in the high-dimensional RKHS
- **KLMS algorithm** is the LMS within the RKHS short notation: $\mathbf{f}(n) = \mathbf{f}(\mathbf{x}(n))$

$$\mathbf{w}(0) = \mathbf{0}$$

$$e(n) = \tilde{e}(n) = t(n) - \mathbf{w}(n-1) \cdot \mathbf{f}(n), n \in [N]$$

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \eta e(n) \mathbf{f}(n)$$

- Transform vector $\mathbf{f}$ of kernel eigenfunctions is only implicitly known $\rightarrow$ iterative estimation:

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \eta e(n) \mathbf{f}(n) = \mathbf{w}(n-2) + \eta e(n-1) \mathbf{f}(n-1) + \eta e(n) \mathbf{f}(n)$$

$$\mathbf{w}(n) = \mathbf{w}(n-2) + \eta(e(n-1) \mathbf{f}(n-1) + e(n) \mathbf{f}(n))$$

$\vdots$

$$\mathbf{w}(n) = \mathbf{w}(0) + \sum_{i=1}^n e(i) \mathbf{f}(i) \quad (5.27)$$

- The AF output is calculable solely with inner products

$$y(n) = \mathbf{w}(n) \cdot \mathbf{f}(n) = \left[\mathbf{w}(0) + \eta \sum_{i=1}^n e(i) \mathbf{f}(i) \right] \cdot \mathbf{f}(n) = \eta \sum_{i=0}^n e(i) \mathbf{f}(i) \cdot \mathbf{f}(n)$$

$$y(n) = \eta \sum_{i=0}^n e(i) \kappa(\mathbf{x}(i), \mathbf{x}(n))$$
(5.28)

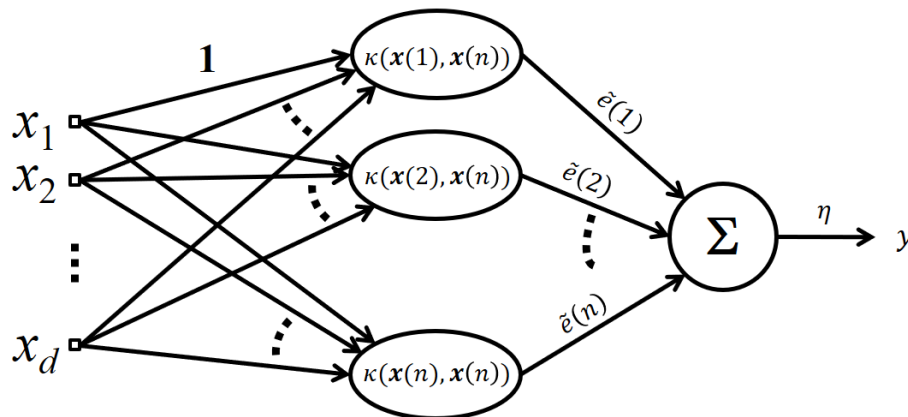
- Sequential learning rule for nonlinear mappings at discrete time n

$$y(n-1) = \eta \sum_{i=1}^{n-1} e(i) \kappa(\mathbf{x}(i), \mathbf{x}(n))$$

$$e(n) \approx \tilde{e}(n) = t(n) - y(n-1)$$

$$y(n) = y(n-1) + \eta e(n) \kappa(\mathbf{x}(n), \mathbf{x}(n))$$
(5.29)

- New kernel functions are inserted for every new training example, $\mathbf{x}(n)$ is the kernel centroid and $\eta e(n)$ is a weight for the influence of $\kappa(\mathbf{x}(n), \mathbf{x}(n))$
- **KLMS** can be regarded as an **RBF network** with the following topology



Similarities to RBF networks, but three essential differences

- Output weights are scaled prediction errors
 - Incremental network by n
 - Kernel function κ is not restricted to RBF, but to Mercer kernel functions
- KLMS is relatively simple: $\mathcal{O}(n)$ operations per filter application and per weight update

Summary of the KLMS algorithm

1. Initialisation $\eta, \kappa, a_1 = \eta t(1), C(0) = \emptyset$
2. Calculation for $n = 1, 2, \dots, N$
 - 2.1. Output calculation $y_{n-1}(\mathbf{x}(n)) = \sum_{i=1}^N a_i \kappa(\mathbf{x}(i), \mathbf{x}(n))$
 - 2.2. Error calculation $\tilde{e}(n) = t(n) - y_{n-1}(\mathbf{x}(n))$
 - 2.3. Centroid saving $C(n) = \{C(n-1), \mathbf{x}(n)\}$
 - 2.4. Coefficients saving $a_n = \eta \tilde{e}(n)$

Problems

- Determining kernel type κ , kernel size σ , and step size η requires experience and several trials
- In general: optimal parameters depend on the joint distribution $p(\mathbf{x}, t)$ and thus on the given data set
- Increasing memory need and numerical expense in case of very long online tasks
- Determining the **kernel type**
 - Bell-shaped kernels are generally appropriate
 - RKHS kernels supply complete diversity
 - Gaussian kernel functions have universal approximation properties and are numerically stable
 - Polynomial kernel functions with finite order are not in RKHS and thus have less diverse solutions
- Determining the **kernel width** σ or γ : $\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{x}'\|^2\right) = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$
 - If σ is too large: all data look as the same in RKHS (inner product $\rightarrow 1$), reduction to linear regression
 - If σ is too small: all data appear different in RKHS (inner product $\rightarrow 0$), no inference for unseen data

- Determining the **step size** η
 - Trade-off between convergence and error adaptation
 - Upper bound: reciprocal value of the largest eigenvalue of the auto-correlation matrix
 - Stability condition: $\eta = \frac{1}{\xi_{max}} < \frac{N}{\sum_{n=1}^N \kappa(x(n), x(n))}$ (ξ_{max} is the largest eigenvalue of $\mathbf{R}_F = \frac{1}{N} \mathbf{F} \mathbf{F}^T$)
- **Misadjustment** m can be estimated with Gram matrix $\mathbf{K}_F = \mathbf{F}^T \mathbf{F}$: $m = \frac{1}{2} \eta \text{tr}(\mathbf{R}_F) = \frac{1}{2N} \eta \text{tr}(\mathbf{K}_F)$
- **Novelty criterion for sparsification**
 - Stationary process environment $\rightarrow$ convergence of the learning algorithm is highly probable
 - But: filter size increases with every time step
 - Sparse models: less memory need, less CPU load, and higher generalisation ability
 - Way out: online sparsification or sequential sparsification with novelty criterion
 - Starting with empty set; insertion of data if novelty criterion is fulfilled $\rightarrow$ code book C
 - Distance δ between input vector and code book $C(n) = \{\mathbf{c}_k\}$: $\delta(n+1) = \min_{\mathbf{c}_k \in C(n)} \|\mathbf{x}(n+1) - \mathbf{c}_k\|$
 - Insertion of $\mathbf{x}(n+1)$ in C : $C(n+1) = \{C(n), \mathbf{x}(n+1)\}$
under the following two conditions: $\delta(n+1) > \delta_{th1}$, $\bar{e}(n+1) > \delta_{th2}$
 - Estimates of both thresholds: $\delta_{th1} \approx \frac{1}{10} \sigma$, $\delta_{th2} \approx \sqrt{\mathbb{E}(e^2(n))} = RMSE$

Self-regularisation property of the KLMS algorithm

- Regularisation plays a central role to gain high generalisation ability
- Regularised cost functions must be defined
- KLMS does not require explicit regularisation because it already exists implicitly
- For the general regression model in RKHS [Haykin, 2002] has proved a robustness theorem which is fulfilled in the RKHS if the model error sequence $v(n)$ has finite energy, i.e. $\sum_i |v(i)|^2 < \infty$
- If this is fulfilled, then the prediction error is bounded by:

$$\|\mathbf{e}\|^2 < \frac{1}{\eta} \|\mathbf{w}^*\|^2 + 2\|\mathbf{v}\|^2, \quad \mathbf{e} = (e(1), e(2), \dots, e(N))^T, \quad \mathbf{v} = (v(1), v(2), \dots, v(N))^T$$

- Then, the norm of the KLMS solution is also bounded

$$\|\mathbf{w}(N)\| < \sqrt{N \xi_{max} \eta (\|\mathbf{w}^*\|^2 + 2\eta \|\mathbf{v}\|^2)}$$

which means compactness of the hypothesis space and high algorithmic stability

- Solution by SVD (singular value decomposition)

- Self-regularisation properties come from different convergence rate along different eigenfunctions
- Assume that $\mathbf{F}$ has the following SVD where r is the rank of $\mathbf{F}$, and $\mathbf{P}, \mathbf{Q}$ are orthogonal matrices, and $\mathbf{S}$ is a diagonal matrix containing the singular values $s_1, s_2, \dots, s_r$
- SVD of $\mathbf{F}$:

$$\mathbf{F} = \mathbf{P} \begin{bmatrix} \mathbf{S} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{Q}^T, \quad \mathbf{S} = \text{diag}(s_1, s_2, \dots, s_r) = \begin{bmatrix} s_1 & 0 & \dots & 0 \\ 0 & s_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & & s_r \end{bmatrix} \quad (5.30)$$

- Without loss of generality the following holds:

$$\mathbf{R}_F = \mathbf{P} \begin{bmatrix} \frac{1}{N} \mathbf{S}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{P}^T, \quad \mathbf{K}_F = \mathbf{Q} \begin{bmatrix} \mathbf{S}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{Q}^T \quad (5.31)$$

- For the following objective function: $J(\mathbf{w}) = \|\mathbf{t} - \mathbf{F}^T \mathbf{w}\|^2 = \|\mathbf{w}^{*T} \mathbf{f}(n) + v(n) - \mathbf{F}^T \mathbf{w}\|^2$
a minimum is found by the pseudo-inverse of $\mathbf{F}$ which leads to the following solution:

$$\mathbf{w}_{PI} = \mathbf{P} \text{diag}(s_1^{-1}, s_2^{-1}, \dots, s_r^{-1}, 0, \dots, 0) \mathbf{Q}^T \mathbf{t} \quad (5.32)$$

- This solution might eventually be ill-conditioned, especially in case of a complex data distribution, of small data sets, or of high noise
- Then, the Tikhonov regularisation (1977) is an alternative by introducing a penalty term in the objective function of the LS approach: $J(\mathbf{w}) = \|\mathbf{t} - \mathbf{F}^T \mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2$. Their solution is as follows:

$$\mathbf{w}_{TR} = \mathbf{P} \text{diag}\left(\frac{s_1}{s_1^2 + \lambda}, \frac{s_2}{s_2^2 + \lambda}, \dots, \frac{s_r}{s_r^2 + \lambda}, 0, \dots, 0\right) \mathbf{Q}^T \mathbf{t} \quad (5.33)$$

- Another alternative is the **restricted RLS solution** if small values s_i lead to high components in $\mathbf{w}_{TR}$
- Smallest values s_i should be reset to zero as this is done in PCA (principal component analysis):

$$\mathbf{w}_{PCA} = \mathbf{P} \text{diag}(s_1^{-1}, s_2^{-1}, \dots, s_k^{-1}, 0, \dots, 0) \mathbf{Q}^T \mathbf{t}, \quad k < r \quad (5.34)$$